

lab3_jupyter_webscraping

May 8, 2026

Webscraping in R

Estimated time needed: **15** minutes

0.1 Objectives

After completing this lab you will be able to:

- Understand HTML via coding practice
- Perform basic webscraping using rvest

Table of Contents

`<ul>`

`<li><a href="#Overview-of-HTML">Overview of HTML</a></li>`

`<li><a href="#The-rvest-library">The rvest library</a></li>`

Overview of HTML

- HTML stands for Hypertext Markup Language and it is used mainly for writing web pages.
- An HTML page consists of many organized HTML nodes or elements that tell a browser how to render its content.
- Each node or element has a start tag and an end tag with the same name and wraps some textual content.

One key feature of HTML is that nodes can be nested within other nodes, organizing into a tree-like structure like the folders in a file system. Below is a basic HTML node structure:

- `<html>` node is the root node,
- `<html>` node has two children: `<head>` and `<body>`.
- Since the `<head>` and `<body>` nodes have the same parent `<html>` node they are siblings to each other.
- Similarly, the `<body>` node has two child nodes, the `<h1>` and `<p>` nodes.

It is important to understand this tree-structure when writing a new HTML page or extracting data from an existing HTML page.

The **revest** library

The **rvest** package is a popular web scraping package for R. After rvest reads an HTML page, you can use the tag names to find the child nodes of the current node.

```
[1]: library(rvest)
```

We also need to import httr library to get some HTML pages by sending HTTP GET request

```
[ ]: library(httr)
```

First let's warm-up by reading HTML from the following character variable `simple_html_text`

```
[2]: # A simple HTML document in a character variable
simple_html_text <- "
<html>
  <body>
    <p>This is a test html page</p>
  </body>
</html>"
```

Then use the `read_html` function to create the HTML root node, i.e., the `html` node by loading the `simple_html_text`

```
[3]: # Create the root html node by reading the simple HTML string
root_node <- read_html(simple_html_text)
root_node

{html_document}
<html>
[1] <body>\n      <p>This is a test html page</p>\n    </body>
```

You can also check the type of `root_node`

```
[4]: class(root_node)
```

1. 'xml_document' 2. 'xml_node'

You can see the class is `xml_node` because `rvest` load HTML pages and convert them using XML format internally. XML has similar parent-child tree structure but more suitable for storing and transporting data than HTML.

Next, let's try to create a HTML node by loading a remote HTML page given a URL

```
[5]: ibm_html_node <- read_html("http://www.ibm.com")
ibm_html_node

{html_document}
<html lang="en">
[1] <head>\n<meta http-equiv="Content-Type" content="text/html; charset=UTF-8 ...
[2] <body class="content-page page basicpage publish " id="content-page-c8104 ...
```

Sometimes you want to download some HTML pages and analyze them offline, you could use `download.file` to do so:

```
[6]: # download the R home page and save it to an HTML file locally called r.html
download.file('https://www.r-project.org', destfile = 'r.html')
```

```
[7]: # Create a html_node from the local r.html file
html_node <- read_html('r.html')
html_node

{html_document}
<html lang="en">
[1] <head>\n<meta http-equiv="Content-Type" content="text/html; charset=UTF-8 ...
[2] <body>\n    <div class="container page">\n        <div class="row">\n            ...
```

Coding Exercise: in the code cell below, download a html node using any URL you like.

```
[10]: # Write your code below. Don't forget to press Shift+Enter to execute the cell
# Example URL
url <- "https://quotes.toscrape.com/"

# Read the webpage
page <- read_html(url)

# Download/select an HTML node
# Here we grab the first quote block
quote_node <- page %>%
  html_element(".quote")

# Print the raw HTML of the node
quote_node

# Extract the text content
html_text2(quote_node)

# Save the node as an HTML file locally
writeLines(as.character(quote_node), "quote_node.html")
```

```
{html_node}
<div class="quote" itemscope="" itemtype="http://schema.org/CreativeWork">
[1] <span class="text" itemprop="text">"The world as we have created it is a ...
[2] <span>by <small class="author" itemprop="author">Albert Einstein</small>\ ...
[3] <div class="tags">\n                Tags:\n                <meta class="keywords" ...
```

“The world as we have created it is a process of our thinking. It cannot be changed without changing our thinking.”by Albert Einstein (about)\nTags: change deep-thoughts thinking world’

Now you know how to read an HTML page from a character variable, a URL, or a local HTML file. Next let’s see how to parse and extract data from a specific node(s) starting from the root node

```
[11]: simple_html_text <- "
<html>
  <body>
    <p style=\"color:red;\">This is a test html page</p>
  </body>
</html>"
```

```
root_node <- read_html(simple_html_text)
root_node
```

```
{html_document}
```

```
<html>
```

```
[1] <body>\n          <p style="color:red;">This is a test html page</p>\n    < ...
```

Get the `<body>` node by using its parent node `<html>`

```
[12]: # Get the child <body> node from <html> root_node
body_node <- html_node(root_node, "body")
body_node
```

```
{html_node}
```

```
<body>
```

```
[1] <p style="color:red;">This is a test html page</p>
```

You can see it has a child node paragraph `<p>`

Let's get the content of the `<p>`

```
[13]: # Get the child <p> node from its <body> node
p_node <- html_node(body_node, "p")
p_content <- html_text(p_node)
p_content
```

'This is a test html page'

The `<p>` node also has `style` attribute with value `color:red;`, which means we want the browser to render its text using red color. To get an attribute of a node, we can use a function called `html_attr("attribute name")`

```
[14]: # Use the p_node as the first argument to get its attribute
style_attr <- html_attr(p_node, "style")
style_attr
```

'color:red;'

In the code cell below, the downloaded `r.html` file (from <https://www.r-project.org>) has an `<img>` node representing an image URL to R logo image (a relative path on its web server), let's try to find the image URL and download it.

Your need to paste the relative path in `<img>` with the the <https://www.r-project.org> to get the full URL of the image, and use the `GET` function to request the image as bytes in the response

```
[15]: # Write your code below. Don't forget to press Shift+Enter to execute the cell
url <- 'https://www.r-project.org'
html_node <- read_html('r.html')

# Get the image node using its root node
img_node <- html_node(html_node, "img")
```

```
# Get the "src" attribute of img node, representing the image location
img_relative_path <- html_attr(img_node, "src")
img_relative_path

# Paste img_relative_path with 'https://www.r-project.org'
image_path <- paste(url, img_relative_path, sep="")
# Use GET request to get the image
image_response<-GET(image_path)
```

‘/Rlogo.png’

```
Error in GET(image_path): could not find function "GET"
Traceback:
```

Then use writeBin() function to save the returned bytes into an image file.

```
[16]: # Parse the body from the response as bytes
image <- content(image_response, "raw")
# Write the bytes to a png file
writeBin(image, "r.png")
```

```
Error in content(image_response, "raw"): could not find function "content"
Traceback:
```

Now, from the file list on the left, you should be able to find a saved r.png file.

In HTML, many tabluar data are stored in <table> nodes. Thus, it is very important to be able to extract data from <table> nodes and preferably convert them into R data frames.

Below is a sample HTML page contains a color table showing the supported HTML colors, and we want to load it as a R data frame so we can analyze it using data frame-related operations.

```
[17]: table_url <- "https://cf-courses-data.s3.us.cloud-object-storage.appdomain.
↳cloud/IBM-DA0321EN-SkillsNetwork/labs/datasets/HTMLColorCodes.html"
```

Like other HTML nodes, let’s first get the <table> node using html_node function

```
[18]: root_node <-read_html(table_url)
table_node <- html_node(root_node, "table")
table_node
```

```
{html_node}
<table border="1" class="main-table">
 [1] <tr>\n<td>Number </td>\n      <td>Color</td>\n      <td>Color Name</td>\ ...
 [2] <tr>\n<td>1</td>\n      <td style="background:lightsalmon;"> </td>\n      ...
 [3] <tr>\n<td>2</td>\n      <td style="background:salmon;"> </td>\n      <td ...
 [4] <tr>\n<td>3</td>\n      <td style="background:darksalmon;"> </td>\n      ...
```

```

[5] <tr>\n<td>4</td>\n      <td style="background:lightcoral;"> </td>\n      ...
[6] <tr>\n<td>5</td>\n      <td style="background:coral;"> </td>\n      <td> ...
[7] <tr>\n<td>6</td>\n      <td style="background:tomato;"> </td>\n      <td> ...
[8] <tr>\n<td>7</td>\n      <td style="background:orangered;"> </td>\n      ...
[9] <tr>\n<td>8</td>\n      <td style="background:gold;"> </td>\n      <td>g ...
[10] <tr>\n<td>9</td>\n      <td style="background:orange;"> </td>\n      <td> ...
[11] <tr>\n<td>10</td>\n      <td style="background:darkorange;"> </td>\n      ...
[12] <tr>\n<td>11</td>\n      <td style="background:lightyellow;"> </td>\n      ...
[13] <tr>\n<td>12</td>\n      <td style="background:lemonchiffon;"> </td>\n      ...
[14] <tr>\n<td>13</td>\n      <td style="background:papayawhip;"> </td>\n      ...
[15] <tr>\n<td>14</td>\n      <td style="background:moccasin;"> </td>\n      ...
[16] <tr>\n<td>15</td>\n      <td style="background:peachpuff;"> </td>\n      ...
[17] <tr>\n<td>16</td>\n      <td style="background:palegoldenrod;"> </td>\n      ...
[18] <tr>\n<td>17</td>\n      <td style="background:khaki;"> </td>\n      <td> ...
[19] <tr>\n<td>18</td>\n      <td style="background:darkkhaki;"> </td>\n      ...
[20] <tr>\n<td>19</td>\n      <td style="background:yellow;"> </td>\n      <t ...
...

```

You can see the table node in a messy HTML format. Fortunately, you don't need to parse it by yourself, `rvest` provides a handy function called `html_table()` to convert `<table>` node into R dataframe

```

[19]: # Extract content from table_node and convert the data into a dataframe
color_data_frame <- html_table(table_node)
head(color_data_frame)

```

	X1	X2	X3	X4	X5
	<chr>	<chr>	<chr>	<chr>	<chr>
	Number	Color	Color Name	Hex Code#RRGGBB	Decimal Code(R,G,B)
A tibble: 6 × 5	1		lightsalmon	#FFA07A	rgb(255,160,122)
	2		salmon	#FA8072	rgb(250,128,114)
	3		darksalmon	#E9967A	rgb(233,150,122)
	4		lightcoral	#F08080	rgb(240,128,128)
	5		coral	#FF7F50	rgb(255,127,80)

But you could see the table headers were parsed as the first row, no worries, we could manually fix that

```

[20]: names(color_data_frame)

```

```

1. 'X1' 2. 'X2' 3. 'X3' 4. 'X4' 5. 'X5'

```

```

[21]: # Convert the first row as column names
names(color_data_frame) <- as.matrix(color_data_frame[1, ])
# Then remove the first row
data_frame <- color_data_frame[-1, ]

head(data_frame)
names(color_data_frame)

```

	Number <chr>	Color <chr>	Color Name <chr>	Hex Code#RRGGBB <chr>	Decimal Code(R,G,B) <chr>
A tibble: 6 × 5	1		lightsalmon	#FFA07A	rgb(255,160,122)
	2		salmon	#FA8072	rgb(250,128,114)
	3		darksalmon	#E9967A	rgb(233,150,122)
	4		lightcoral	#F08080	rgb(240,128,128)
	5		coral	#FF7F50	rgb(255,127,80)
	6		tomato	#FF6347	rgb(255,99,71)

1. 'Number' 2. 'Color' 3. 'Color Name' 4. 'Hex Code#RRGGBB' 5. 'Decimal Code(R,G,B)'

That's it for **webscraping** in R, there is a lot more you can do check out rvest.

0.2 Authors

Hi, this is Yan Luo the author of this notebook.

I hope you found it easy to learn how to do webscraping in R! Feel free to connect with us if you have any questions.

0.2.1 Other Contributors

##

© IBM Corporation 2021. All rights reserved.

<!-- ## Change Log

| Date (YYYY-MM-DD) | Version | Changed By | Change Description |
|-------------------|---------|------------|-------------------------|
| 2021-03-05 | 1.0 | Yan | Initial version created |

~!>